Proceedings of the 2010 IEEE
International Conference on Robotics and Biomimetics
December 14-18, 2010, Tianjin, China

A New Bug-type Navigation Algorithm Considering Practical Implementation Issues for Mobile Robots

Yi Zhu, Tao Zhang, Jingyan Song, Xiaqin Li

Abstract—**a new Bug-type algorithm is proposed in this paper for the navigation of mobile robots. Unlike the related works which mainly focus on the theoretical analysis while ignoring the implementation issues, the new method presents not only an abstract concept which shortens the path length comparing with some previous works in the premise of guaranteeing convergence by its new switching conditions, but also a lower layer approach to realize its concept by keeping a safe distance to obstacles for collision avoidance. Simulation studies show that the new method generates shorter path than the classical Bug2 algorithm and a recent work termed as Bug2+. Experiment results on a real Pioneer3-AT robot further verify its practicability.**

I. Introduction

AUTONOMOUS navigation of mobile robots has received considerable attention in recent twenty years. The main task of a navigation algorithm designed for mobile robots is to plan the motion of the robot for avoiding obstacles and approaching the goal. Previous works for achieving this object fall into two categories: global or local planning. However, both the two types have their own drawbacks.

The usual concept of the global planning is to plan a path connecting the start point and the goal by searching a graph which models the connectivity of the global environment. In early works [1], this graph is constructed off line based on the assumption that the environment is completely known. To enable the implementation on line, more recent works [2]-[4] focus on incrementally building the global graph model by local sensory information. The global planning methods are usually complete or convergent, which implies that they will always find a path connecting the start point and the goal if such a path exists, otherwise they will report fail. Nevertheless, the reliability of a global planning method depends on an accurate graph model which is often not available in advance for many application scenarios and also not easy to acquire on line due to the uncertainties of practical sensors. Furthermore, the construction of the model leads to expensive computation and the complexity of implementation.

In contrast, local planning methods directly map the local sensory information to the control commands of the robot in every control cycle. Algorithms for realizing such a mapping include potential field method [5] and its derivative algorithms [6]-[8], velocity space methods [9], [10], fuzzy logic methods [11], and the recent method named nearness diagram navigation [12]. Without build a global model, these methods are usually simple to implement and suitable for real-time applications due to their reactive property. However, they do not guarantee convergence. Since there is no global model, greedy search strategies are adopted in all the methods mentioned above to select the next direction of the robot. Hence the robot may be trapped in a local minimum before reaching the goal by using these methods.

Since the global methods are difficult to implement while the local methods do not guarantee convergence, this paper considers a special algorithm family named the Bug algorithms [13] which do not have both these limitations. In the Bug algorithms, the motion of the robot always switches between two modes: moving toward the goal unless an obstacle is encountered, otherwise following the boundary of the obstacle until a leaving condition is satisfied. The Bug algorithms are similar to local methods since they need only local environment information within a range from the robot. The difference is that the leaving condition of boundary following mode is design to guarantee convergence of the algorithm by ensuring that the distance to the goal decreases between any two successive switching positions.

Nevertheless, The Bug algorithms also have limitations. Based on the assumption that the robot is a point moving in a two-dimensional plane, the previous works focus mainly on designing new leaving conditions to optimize the path length (in the premise of guaranteeing convergence) in an abstract view. However, implementation issues are usually ignored. For example, the published works have seldom illustrated in detail that how to properly realize the two motions based on practical sensors in real world [13]. Additionally, the robot is usually assumed to maintain close to an obstacle without considering collision avoidance when it follows along the obstacle's boundary, but it is not safe if the robot moves fast. Hence a new bug-type algorithm considering such practical issues is proposed in this paper. The new algorithm has two main contributions. Firstly, comparing with some previous Bug algorithms, it shortens the path length in the premise of guaranteeing convergence to the goal due to the new switching conditions between the motion modes. Secondly, unlike the previous Bug algorithms that always ignore the practical implementation issues, the new algorithm requires the robot to keep a safe distance to obstacles for collision avoidance, while we propose a lower layer method termed as the distance histogram (DH) method to generate control

Manuscript received July 23, 2010. This work was supported by Aviation Industry Corporation of China under Aviation Science Funds 20080758003

Yi Zhu, Tao Zhang, Jingyan Song and Xiaqin Li are with Department of Automation, Tsinghua University, Beijing, 100084, China. (e-mail: zhu-y07@ mails.tsinghua.edu.cn).

978-1-4244-9318-0/10/$26.00 © 2010 IEEE

commands on line based on range sensors for realizing this concept and both the two motion modes.

The next section introduces the proposed algorithm in the abstract view and its properties are analyzed in Section III. Then we illustrate how to realize the proposed algorithm based on range sensors by our DH method in Section IV. In Section V, the results of simulations and experiments on a Pioneer3-AT robot are presented.

II. Algorithm Description

To formulate the algorithm, two definitions which will be used in this paper and the assumptions of the environment and the robot are firstly presented here.

Definition 1 The number of the obstacles in an environment is "finite" if and only if any disc of finite radius in the environment intersects a finite set of obstacles.

Definition 2 A path is "admissible" if and only if the distance between any point of this path and its nearest obstacle is no less than a predefined safe value D.

Assumption 1 The environment is a two-dimensional plane populated by finite stationary obstacles whose boundaries are arbitrary piecewise-smooth curves with finite length.

Assumption 2 The robot is a disc equipped with range sensors (limited by a constant detection range). The detectable area is at least a sweep of $180°$ in the front of the robot.

Note that a safe distance can be set for other shaped robots to imitate a disc. Next we describe the concept of the proposed algorithm firstly and its pseudo code later.

Similar to the classical Bug algorithms [13], the proposed algorithm navigates the robot by two motion modes: moving towards the goal and boundary following. The difference is a safe distance D which is added to avoid obstacles based on the range sensory information and the new switching conditions which shortens the path length comparing with some previous Bug algorithms.

Initially, the robot moves toward the goal. In this mode, the robot always moves in the direction θ that has the minimal bias to the current goal direction θ_g (All the angles in this paper refer to the local coordinates where the original angle equals to the head direction of the robot and anticlockwise direction is positive) and satisfies:

$$\left|\theta - \theta_g\right| < 90° \wedge \theta \in \Theta_{safe} \tag{1}$$

where "$\wedge$" represents "logic and", Θ_{safe} is the current set of all the safe θ that the robot can at least keep a distance D to the nearest obstacles if this direction is selected. If no θ satisfies (1), which implies that the robot can't find any θ which shortens the distance to the goal while maintain safe for collision avoidance, it must exist an obstacle that prevents the robot from approaching the goal any more. Such a position is termed as a hit point. Then the robot starts to follow the boundary of this obstacle. Before switching to the boundary following mode, a following direction, i.e., being left or right to the obstacle, is selected and it is recorded with the position of the current hit point in an information list termed as hit list. Meanwhile, the minimal goal distance $d_{\min}$ is updated by the distance from the current hit point to the goal.

In the boundary following mode, the robot moves along the boundary without changing the direction and maintains a safe range D to the obstacle. During the motion, the robot updates its minimal goal distance $d_{\min}$ and switches back to the mode of moving toward the goal once the state of the robot satisfies:

$$\begin{cases} 0° < \theta_g < 180° \wedge d_{\text{cur}} < d_{\min} & \text{if } dir = \text{R} \\ -180° < \theta_g < 0° \wedge d_{\text{cur}} < d_{\min} & \text{if } dir = \text{L} \end{cases} \tag{2}$$

where d_{cur} is the current goal distance while $d_{\min}$ is the minimal goal distance up to the last control cycle, dir is the following direction, L represents "left to the obstacle" and R represents "right to the obstacle". Such a position is termed as a leave point. Furthermore, the following direction will be reversed if the robot reaches a previous hit point (not the current hit point, i.e., the start point of the current boundary following motion) in the same direction as the record in the hit list, while this record will be deleted. This addition will be used to strictly guarantee the algorithm convergence as shown in Section III. The above description is summarized in the form of pseudo code in the following.

Mode 1 (motion toward the goal) Move in the best direction (having the minimal bias to θ_g) satisfying (1), until one of the following events occurs:

a) the goal is reached. Stop.

b) there is no θ satisfies (1). Select a following direction and record it with the current position in the hit list. Update $d_{\min}$ by the current goal distance. Switch to Mode 2.

Mode 2 (boundary following) Move along the boundary of the nearest obstacle in the selected following direction and update $d_{\min}$ in the motion, until one of the following events occurs:

a) the goal is reached. Stop.

b) a leave point satisfying (2) is reached. Switch to Mode 1.

c) a previous hit point is reached in the same direction as the record in the hit list. Reverse the following direction and delete this record. Continue Mode 2.

d) the current hit point is reached without event c occurs after the last time that the robot passed the current hit point. The goal is unreachable. Stop.

Fig. 1 (S represents the start point, G is the goal, H is the hit point, L is the leave point and the solid line is the generated path) presents an example comparing the proposed algorithm with the classical Bug2 algorithm [14]. In Bug2, the robot always moves directly toward the goal until it hits an obstacle. Then it maintains close to this obstacle and moves along its boundary until the robot again reaches the M-line (the line connecting the start point and the goal) while the line section connecting the robot and the goal doesn't intersect the obstacle at the current position. Comparing with Bug2, the proposed algorithm keep a safe range to obstacles for

collision avoidance and its hit points and leave points are different. Intuitively, it will shorten the path length since it navigates the robot leave the obstacle before reaching the M-line. The test result of this property will be presented later in Section V. We will firstly analyze its convergence property in the next section and show how to realize it in Section IV.

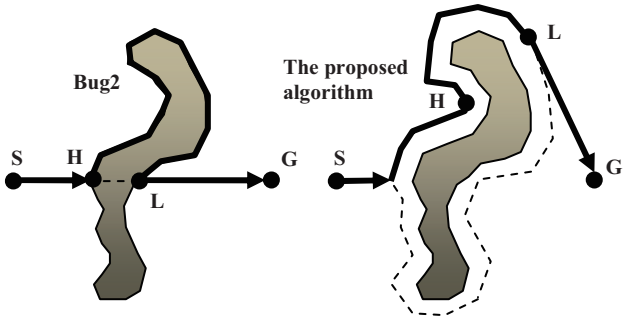


Fig. 1. An example comparing the proposed algorithm with Bug2.

III. Algorithm Analysis

Before the analysis, we introduce several denominations. By moving along an obstacle while keeping a safe distance, the robot can generate several continuous trajectories (if the obstacle doesn't enclose the goal, there is only one, otherwise there are several). We term the trajectories corresponding to the same obstacle as the enlarged boundary of this obstacle. An enlarged boundary can be further divided into several parts belonging to two types: H-line and L-line. A H-line is the part that any line section connecting its point and the goal intersects the enlarged boundary $2n$ times, while a L-line does that $2n-1$ times. n is any positive integer. (An example is shown in Fig. 2). It can be seen that only position in H-line is possible to become a hit point in a motion toward the goal, while only position in L-line is possible to become a leaving point satisfying (2) in a boundary following motion.

Some lemmas are firstly presented in the following and the convergence property of the algorithm is proved later.

Lemma 1 The goal distance monotonically decreases at each hit point.

Proof After reaching every hit point H_1, the robot switches to boundary following motion and resume back to the motion toward the goal if a leave point L_1 satisfying $d_{H_1} > d_L$ according to (2) is reached. d_X represents the goal distance at point X. Then in the next motion toward the goal, the robot moves in a direction θ whose bias to θ_g is less than $90°$ according to (1). Therefore the goal distance always decreases until the next hit point H_2 is reached. Hence $d_{H_1} > d_L > d_{H_2}$. □

Lemma 2 After encountering a hit point, the robot will always reach a leave point later in the motion of boundary following if an admissible path connecting the start point and the goal exists in the environment.

Proof Since the leaving point always occurs in a L-line, we focus on all the L-lines of an obstacle. As shown in Fig.2, obviously, if an admissible path connecting the start point and the goal exists, there must have an reachable L-line that any line section connecting its point and the goal intersects the entire enlarged boundary only once, which indicates that as to any hit point H, we can always find a corresponding leave point L in this L-line satisfying (2), e.g., the point that belongs to both Line Section HG and this L-Line (note that the robot may leave the obstacle in the other point of this L-line or other L-lines before reaching this point). □

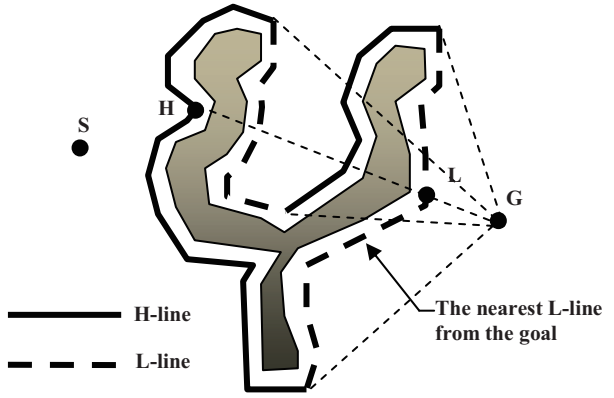


Fig. 2. An example illustrating the H-line and the L-line.

Corollary 1 If the current hit point is reached again in the boundary following mode without reversing the following direction after the last time that the robot passed the current hit point, there doesn't exist an admissible path connecting the start point and the goal, i.e., the goal is unreachable.

Proof If the situation mentioned above happens, it implies that the robot has completed a loop around the obstacle without find a leave point. According to Lemma 2, the goal must be unreachable. □

Lemma 3 Guided by the proposed algorithm, the robot will meet finite hit points before arriving at the goal or reporting fail.

Proof In the proposed algorithm, if the robot meets a hit point, it will record it in the hit list. When the robot meets this hit point again, it will moves in the opposite following direction as the record in the list later. Hence the robot can go through the entire H-line corresponding to this hit point by reaching a hit point at most twice, while it records the minimal goal distance in this H-line by updating $d_{\min}$. Then the robot will never meet this point or other points of this H-line again in the motion toward the goal according to Lemma 1 (the points of this H-line may be met in a boundary following motion later, but they will not become a hit point again). Therefore the robot will meet only finite hit points before arriving at the goal or reporting fail, since the number of the H-lines is finite due the finite length of the piecewise-smooth enlarged boundaries. □

Theorem 1 The proposed algorithm always guides the robot to reach the goal if an admissible path connecting the start point and the goal exists in the environment.

Proof According to Lemma 2, the robot will always find a leave point after meeting a hit point if the goal is reachable, while the hit points are finite according to Lemma 3. Hence the robot can't always switch between the two modes and it will finally reach the goal. □

IV. Algorithm Realization

Besides the new switching conditions, another main contribution of this paper is that we propose the DH method to realize our concept, while the realization approach is also valid for many other Bug algorithms with some revision. The concept of the DH method originates from the VFH method which is a local planning method proposed by Borenstein and Koren [7]. Next we briefly review the VFH method at first and illustrate the improvement in the DH method later. At last we show that how to realize our algorithm by the DH method.

A. The VFH Method

As shown in Fig. 3, the VFH method [7] divides the $360°$ directions around the robot into several safe sectors that the obstacle density (a value that is proportional to the negative of the distance from the robot to obstacles) in any direction of this sector is no less than a threshold. The middle direction of every safe sector is a candidate and a best one that has the minimal bias to the goal direction is selected as the final motion direction from them. The VFH method can generate smooth trajectory while keeping a safe distance to obstacles at a relatively high speed. Furthermore, it can guide the robot to go through narrow corridors. However, the VFH method is a local planning method without guaranteeing its convergence.

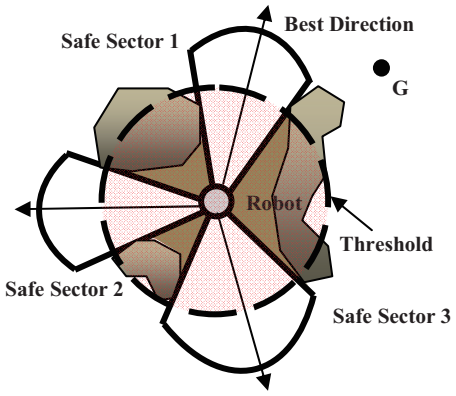


Fig. 3. An example illustrating the concept of the VFH method.

B. The DH Method

We improve the VFH method in three aspects. Firstly, The DH method directly compares the distance data of the range sensors with a threshold to judge whether a direction is safe without constructing a density function based on a grid map as the VFH method does since it is unnecessary for our purpose of realizing a Bug-type algorithm and it is also computational expensive. We redefine the safe sector as a continuous sector that the obstacle distance d_θ in any direction θ of this sector is no less than a threshold d_s ($d_s = 0.4\text{m}$ in our experiments), while the width of the sector is larger than a predefined safe angle θ_s ($\theta_s = 120°$ in our experiments). Therefore the robot will try to keep at least a distance of $d_s \sin(\theta_s / 2)$ from obstacles. Note that in our definition, a wide continuous sector that all the directions of it are safe can be divided into several safe sectors while an overlap can exist between any two safe sectors of it. Secondly, the VFH method controls the speed of the robot by a simple law which keeps the speed proportion to the obstacle distance in the current motion direction of the robot. We substitute it in the DH method by our speed control method [16] with a little revision (it hasn't been described in detail here since it isn't the main work of this paper and the space is limited) that considers more factors of the environment and the state of the robot. The third also the key improvement is that in the DH method, how to select the final motion direction from the safe sectors depends on the states of the robot. Therefore we can realize the two motion modes described in Section II by designing proper states and smooth transition among them, which is illustrated in detail in the following.

C. Realization of the Proposed Algorithm

Four seamless states are designed for realizing the two motion modes of the proposed algorithm, while one for the motion toward the goal and three for the motion of boundary following. The rules of selecting the motion direction in these states are illustrated respectively in the following, while the diagram of state transition is shown in Fig. 4.

State 1 (moving toward the goal) The robot moves in a direction θ which is the middle direction of a safe sector with the minimal bias (at most $90°$) to θ_g. It can be formulated as:

$$\begin{aligned} \theta &= \arg\min_{\theta \in \Theta_1} \left\{ \left| \theta - \theta_g \right| \right\} \\ \Theta_1 &= \left\{ \theta \,|\, \left|\theta - \theta_g\right| < 90° \wedge \left\{ \theta' \,|\, \left|\theta' - \theta\right| < \frac{\theta_s}{2} \right\} \subset \bar{\Theta}_s \right\} \\ \bar{\Theta}_s &= \left\{ \theta \,|\, (\left|\theta - \theta_g\right| < \frac{\theta_s}{2} \vee -90° < \theta < 90°) \wedge d_\theta > d_s \right\} \end{aligned} \tag{3}$$

where $\bar{\Theta}_s$ is the extended set of all the detectable safe directions. It's termed as the extended set since that besides the sweep of $180°$ in the front of the robot, the directions deviating the goal less than $\theta_s / 2$ is always considered to be detectable and d_θ of these directions is assumed to be larger than d_s if it's not in the detectable area. The extender set can guide the robot turning to the goal direction to detect whether it is safe if the goal direction is not detectable before this turning. If $\Theta_1 = \varnothing$, it indicates occurs, then the robot firstly switch to a transition state before the boundary following mode. In this transition state, the robot turns to the goal direction and move forward to target the obstacle preventing it from the goal until satisfies (condition 1 in Fig. 4):

$$\begin{aligned} \Theta_2 &= \left\{ \theta \,|\, \left\{ \theta' \,|\, \left|\theta' - \theta\right| < \frac{\theta_s}{2} \right\} \subset \Theta_s \right\} = \varnothing \\ \Theta_s &= \left\{ \theta \,|\, -90° < \theta < 90° \wedge d_\theta > d_s \right\} \end{aligned} \tag{4}$$

Then the robot transits to State 2 after finishes the operation described in event b of Mode 1 (described in Section II).

State 2 (rotating without translation) The robot rotates in the same direction at a constant rotational speed ω_0 without translation until $\Theta_2 \neq \varnothing$ (condition 2 in Fig. 4). Then the

robot transits to State 3.

State 3 (moving along a convex boundary) The robot moves along the boundary of the followed obstacle by selecting the nearest safe motion direction to the obstacle, which can be formulated as:

$$\theta = \begin{cases} \arg\min_{\theta \in \Theta_2} \{\theta\} & dir = \mathrm{R} \\ \arg\max_{\theta \in \Theta_2} \{\theta\} & dir = \mathrm{L} \end{cases} \tag{5}$$

However, State 3 can only be used for convex boundary. To move along boundaries of any shape, it should coordinate with State 2. If $\Theta_2 = \varnothing$, the robot should transits to State 2 (condition 3 in Fig. 4) and transits back after $\Theta_2 \neq \varnothing$. If event b of Mode 2 occurs (condition 7 in Fig. 4), the robot finishes the operation described in Section II and switches to State 1 after a transition state in which the robot rotates its direction to the goal direction. If event c of Mode 2 occurs (condition 4 in Fig. 4), the robot transits to State 4 for reversing its following direction. If event d of Mode 2 occurs, the robot stops and report fails.

State 4 (reversing the following direction) The robot rotates in the same direction at a constant rotational speed ω_0 without translation until it can smoothly transit to other states. The robot transits to State 2 if $\Theta_2 = \varnothing$ (condition 5 in Fig. 4), while it transits to State 3 if the robot has rotated over $180°$ (condition 6 in Fig. 4).

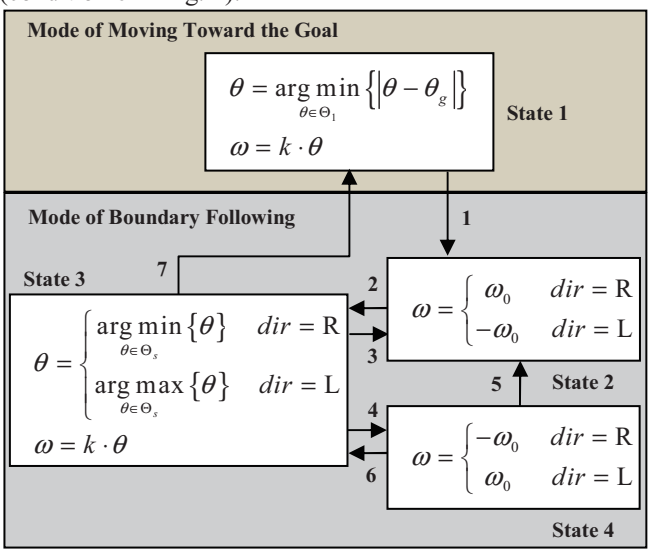


Fig. 4. The state transition diagram for realizing the proposed algorithm.

Such a concept can also be utilized for realizing many other Bug algorithms, e.g., Bug2, by simply instituting condition 1 and condition 7 in Fig. 4 by the switching conditions of the corresponding Bug algorithm (State 4 should be deleted since other Bug algorithms don't need reverse the direction).

V. Experiment Results

The experiment study of the proposed algorithm consists of simulations based on MobileSim that is the special simulation software for Pioneer robot and experiments on a real Pioneer3-AT robot.

A. Simulation Studies

Based on the model of a Pioneer3-AT robot in MobileSim, the validity of the proposed algorithm has been tested in two complex virtual environments as shown in Fig. 5 by 144 times. The start and goal points are randomly selected (the pairs within obstacles are deleted), while the laser sensor that can detect a sweep of $180°$ in the front of Pioneer3-AT is used as the range sensor in both simulations and experiments in a real world. In all the simulations, the robot has reached the goal, which verifies the convergence of the proposed algorithm.

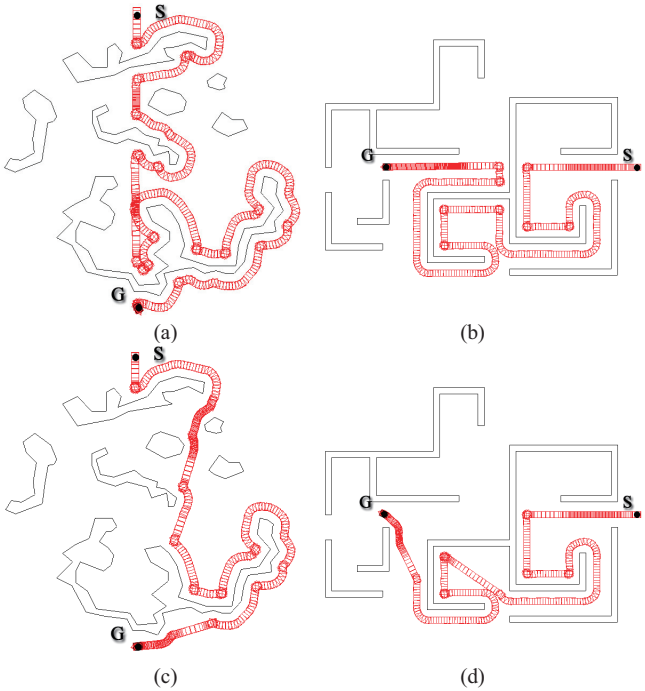


Fig. 5. Simulation results of Bug2+ and the proposed algorithm: (a) Bug2+ in environment 1 populated by scattered obstacles; (b) Bug2+ in environment 2 that is a maze; (c) the proposed algorithm in environment 1; (d) the proposed algortihm in environment 2.

For comparison, the classical Bug2 algorithm (described in Section II) and a recent work terms as Bug2+ [15] which is an improved editions of Bug2 are also respectively simulated 144 times in the same sample of the proposed algorithm. The difference between Bug2 and Bug2+ is that Bug2 only updates $d_{\min}$ at a hit point, while Bug2+ updates $d_{\min}$ if the robot reaches the M-line in the boundary following motion and it doesn't satisfy the leaving condition. Due to the revision, Bug2+ may shorten the path length in some scenarios. Since both Bug2 and Bug2+ haven't illustrated how to realize them, they are realized by our DH method (for the realization, a safe distance to the obstacle is added to the original Bug2 and Bug2+). Additionally, to unify the conditions of the three algorithms, the robot always selects "right to the obstacle" as its following direction. The simulation results of the average path length generated by the three algorithms are shown in Table I, while two pairs of results comparing Bug2+ with the proposed algortihm is presented in Fig. 5 as an example.

TABLE I
AVERAGE PATH LENGTH GENERATED BY THE THREE BUG ALGORITHMS

Environment	Bug2	Bug2+	Our algorithm
Scatter Obstacles	1.96	1.95	1.90
Maze	2.48	2.47	1.95

The average path length is presented as the relative value to the globally shortest admissible path.

Table I shows that the proposed algorithm shortens the path length comparing with Bug2 and Bug2+, while the reason is that it usually leaves the followed obstacle earlier than the other two algorithms which can be seen in Fig. 5.

B. *Experiments on a Real Pioneer3-AT Robot*

To further test the practicability of the proposed algorithm, the algorithm is implemented on a real Pioneer3-AT robot (shown in Fig. 6) while plenty of experiments have been carried out in an office environment populated by tables, chairs and various boxes.

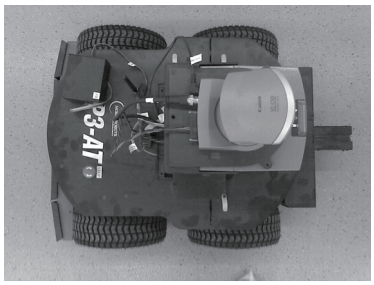
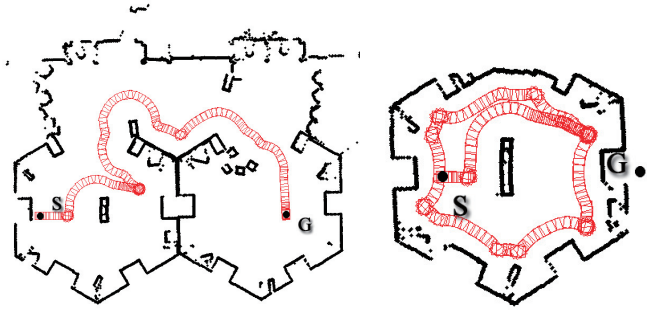


Fig. 6. The Pioneer3-AT robot used in the experiment.

Fig. 7 presents two results of them. In Fig. 7(a), the robot finally reaches the goal in a complex environment while generating a smooth trajectory and always keeping a safe distance to obstacles. In Fig. 7(b), the robot moves in a closed environment. After a loop, it is aware of the fact the goal is unreachable. Then it stops and reports fail. The highest speed of the robot is up to about $0.6\mathrm{m/s}$ in the experiments.



(a) (b)

Fig. 7. Experiment results of the proposed algorithm: (a) the robot reaches the goal in a complex environment; (b) the robot stops and reports fail once detecting that the goal is unreachable.

VI. CONCLUSION

Comparing with many other navigation algorithms for mobile robots, the Bug algorithms have many advantages. For example, they are simple but convergent. However, the previous works focus mainly on the theoretical analysis while ignoring the implementation issues. Therefore a new Bug-type navigation algorithm which considers such practical issues is proposed in this paper. Two main contributions are made by the new algorithm. Firstly, it generates shorter path than some previous Bug algorithms, e.g., the classical Bug2 and Bug2+, while guaranteeing convergence. Secondly, it presents a lower layer approach to solve the implementation issues such as the algorithm realization and obstacle avoidance. The validity and practicability of the proposed algorithm are verified by plenty of simulations based on MobileSim and experiments on a real Pioneer3-AT robot.

REFERENCES

[1] J. C. Latombe, *Robot Motion Planning.* Boston, MA: Kluwer Academic Publishers, 1991.

[2] H. Choset, J. W. Burdick, "Sensor based planning, part II: Incremental construction of the generalized Voronoi graph," in IEEE International Conference on Robotics and Automation, Nagoya, Japan, 1995, pp. 1643-1649.

[3] E. Rimon, "Construction of C-space roadmaps from local sensory data. What should the sensors look for?" Algorithmica, 1997, vol. 17, no. 4, pp. 357-379.

[4] Y.-C. Chang, Y. Yamamoto, "Path planning of wheeled mobile robot with simultaneous free space locating capability," Intelligent Service Robotics, 2009, vol. 2, no. 1, pp. 9-22.

[5] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robot," The International Journal of Robotics Research, 1986, vol. 5, no. 1, pp. 90-98.

[6] J. Borenstein J, Y. Koren, "Real-time obstacle avoidance for fast mobile robots," IEEE Transactions on Systems, Man, and Cybernetics, 1989, vol. 19, no. 5, pp. 1179-1187.

[7] J. Borenstein J, Y. Koren, "The vector field histogram – fast obstacle avoidance for mobile robots," IEEE Journal of Robotics and Automation, 1991, vol. 7, no. 3, pp. 278-288.

[8] I. Ulrich, J. Borenstein, "VFH*: local obstacle avoidance with look ahead verification," in IEEE International Conference on Robotics and Automation, San Francisco, USA, 2000, pp. 2505-2511.

[9] R. Simmons, "The curvature-velocity method for local obstacle avoidance," in IEEE International Conference on Robotics and Automation. Minneapolis, USA, 1996, pp. 3375-3382.

[10] D. Fox, W. Burgard, S. Thrun, "The dynamic window approach to collision avoidance," IEEE Robotics and Automation Magazine, 1997, vol. 4, no. 1, pp. 23-33.

[11] B. Beaufrere, S. Zeghloul, "A mobile robot navigation method using a fuzzy logic approach," Robotica, 1995, vol. 13, pp. 437-448.

[12] J. Minguez J, L. Montano, "Nearness diagram (ND) navigation: collision avoidance in troublesome scenarios," IEEE Transactions on Robotics and Automation, 2004, vol. 20, no. 1, pp. 45-59.

[13] J. Ng, T. Braunl, "Performance comparison of bug navigation algorithms," Journal of Intelligent and Robotic Systems, 2007, vol. 50, pp. 73-84.

[14] V. Lumelsky, A. Stepanov, "Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape," Algorithmica, 1987, vol. 2, pp. 403-430.

[15] J. Antich, A. Ortiz, J. Minguez, "A bug-inspired algorithm for efficient anytime path planning," in IEEE/RSJ International Conference on Intelligent Robots and Systems, St. Louis, USA, 2009, pp. 5407-5413.

[16] Y. Zhu, T. Zhang, J. Song, "Path planning for nonholonomic mobile robots using artificial potential field method," Control Theory & Applications, 2010, vol. 27, no. 2, pp. 152-158 (in Chinese).